

Symulator Misji Ratunkowej - Algorytm PSO

Sprawozdanie z projektu

Wydział: Elektryczny

Przedmiot: JIMP

Data: Styczeń 2026

Autorzy: Danila Miadzedz, Nazar Klymash

1. Opis zadania

1.1 Cel projektu

Celem projektu jest stworzenie symulatora "Misji Ratunkowej" z wykorzystaniem algorytmu Particle Swarm Optimization (PSO). Program symuluje rój dronów poszukujących zaginionego turysty posiadającego nadajnik radiowy na nieznanym terenie.

1.2 Scenariusz

Wyobraźmy sobie teren, na którym zaginął turysta posiadający nadajnik radiowy. Sygnał nadajnika jest najsilniejszy w miejscu przebywania turysty, a słabnie wraz z odległością i ukształtowaniem terenu. Drony (cząstki PSO) zostają zrzucone w losowych miejscach na mapie, a ich zadaniem jest jak najszybsze zlokalizowanie źródła sygnału.

2. Algorytm PSO - Podstawy teoretyczne

2.1 Czym jest PSO?

Particle Swarm Optimization to algorytm optymalizacyjny inspirowany naturą - zachowaniem stad ptaków. W przeciwieństwie do algorytmów śledzących pojedyncze rozwiązanie, PSO zarządza populacją (rojem) rozwiązań nazywanych cząstkami.

2.2 Mechanizm ruchu cząstek

Każda cząstka przemieszcza się po przestrzeni. Jej ruch jest wypadkową trzech czynników:

- Bezwładność - cząstka chce utrzymać swój dotychczasowy kierunek ruchu
- Pamięć własna (pBest) - cząstka pamięta najlepsze miejsce, w którym kiedykolwiek była
- Wiedza roju (gBest) - cząstka zna najlepsze miejsce znalezione przez którąkolwiek cząstkę z całego roju

2.3 Wzory matematyczne

W każdej iteracji aktualizujemy prędkość (v) i pozycję (x) każdej cząstki:

$$v_i(t+1) = w \times v_i(t) + c_1 \times r_1 \times (pBest_i - x_i(t)) + c_2 \times r_2 \times (gBest - x_i(t))$$

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Gdzie:

- w - waga bezwładności (np. 0.5)
- c_1, c_2 - współczynniki uczenia (np. 1.0)
- r_1, r_2 - losowe liczby z zakresu $[0, 1]$
- $pBest_i$ - najlepsza znana pozycja danej cząstki
- $gBest$ - najlepsza znana pozycja całego roju

3. Implementacja

3.1 Struktura projektu

Projekt został podzielony na następujące moduły:

Plik	Opis
main.c	Program główny, obsługa argumentów CLI, inicjalizacja
map.c / map.h	Obsługa mapy terenu - wczytywanie, zwalnianie pamięci
pso.c / pso.h	Logika algorytmu PSO, struktury cząstki i roju
logger.c / logger.h	Zapis danych pośrednich do pliku CSV
utils.c / utils.h	Funkcje pomocnicze, generator liczb losowych
Makefile	Skrypt do kompilacji projektu

3.2 Struktury danych

Struktura cząstki (Particle)

```
typedef struct {  
    double x;        // pozycja X  
    double y;        // pozycja Y  
    double vx;       // prędkość X  
    double vy;       // prędkość Y  
    double best_x;    // najlepsza znana pozycja X  
    double best_y;    // najlepsza znana pozycja Y  
    double best_v;    // wartość funkcji celu w pBest  
} Particle;
```

Struktura roju (Swarm)

```
typedef struct Swarm {  
    Particle *particles; // tablica cząstek  
    int num_particles;   // liczba cząstek w roju  
    double global_best_x; // globalne najlepsze X  
    double global_best_y; // globalne najlepsze Y  
    double global_best_value; // wartość funkcji celu w gBest  
} Swarm;
```

3.3 Zarządzanie pamięcią

Program wykorzystuje dynamiczną alokację pamięci dla tablicy cząstek (Particle*) oraz mapy terenu (double**). Wszystkie zasoby są poprawnie zwalniane przed zakończeniem programu.

3.4 Funkcja celu

Cząstki poruszają się w przestrzeni ciągłej (double), a mapa jest dyskretna. Funkcja map_get_value() rzutuje pozycję na indeksy tablicy i obsługuje wyjście poza zakres zwracając wartość karną (-10000.0).

4. Instrukcja użytkowania

4.1 Kompilacja

Korzystając z Makefile:

```
make
```

Albo klasycznie:

```
gcc -Wall -Wextra -std=c99 -O2 -o pso.exe main.c map.c pso.c logger.c
```

4.2 Uruchomienie

```
./pso <plik_mapy> -p <cząstki> -i <iteracje> -c <konfig> -n <zapis> -o  
<plik_wyjściowy>
```

Opcje:

- -p <liczba> Liczba cząstek (domyślnie: 30)
- -i <liczba> Liczba iteracji (domyślnie: 100)
- -c <plik> Plik konfiguracyjny PSO
- -n <liczba> Co którą iterację zapisywać (0 = brak)
- -o <plik> Plik wyjściowy CSV

Przykład:

```
./pso terrain.txt -p 50 -i 200 -c config.txt -n 2 -o output.csv
```

5. Testy działania programu

5.1 Test: Mapa 7x7

Algorytm poprawnie odnalazł maksimum sygnału (27.60) na pozycji (6.03, 2.49) w prawym górnym rogu mapy testowej.

5.2 Wpływ liczby cząstek

Liczba cząstek	Iteracje do znalezienia
10	~50
30	~20
50	~10
100	~5

Wniosek: Większa liczba cząstek przyspiesza zbieżność, ale zwiększa koszt obliczeniowy.

5.3 Wpływ parametrów PSO

w	c1	c2	Zachowanie
0.9	1.0	1.0	Wolna zbieżność, dobre przeszukiwanie
0.5	1.0	1.0	Zbalansowane
0.3	1.5	1.5	Szybka zbieżność, ryzyko min. lokalnego
0.5	2.0	0.5	Większa eksploracja indywidualna
0.5	0.5	2.0	Szybka zbieżność do gBest

6. Wnioski

1. Algorytm PSO skutecznie znajduje globalne maksimum nawet na mapach z wieloma przeszkodami
2. Parametry algorytmu mają znaczący wpływ na wydajność - waga bezwładności kontroluje równowagę między eksploracją a eksploatacją
3. Modułowa struktura projektu ułatwia rozbudowę i testowanie
4. Dynamiczna alokacja pozwala na dowolne rozmiary map i rojów
5. Zapis do CSV umożliwia późniejszą analizę i wizualizację wyników

7. Podział prac w zespole

Osoba	Zakres odpowiedzialności
DM	Implementacja algorytmu PSO, struktury danych
NK	Obsługa map, CLI, testy, dokumentacja